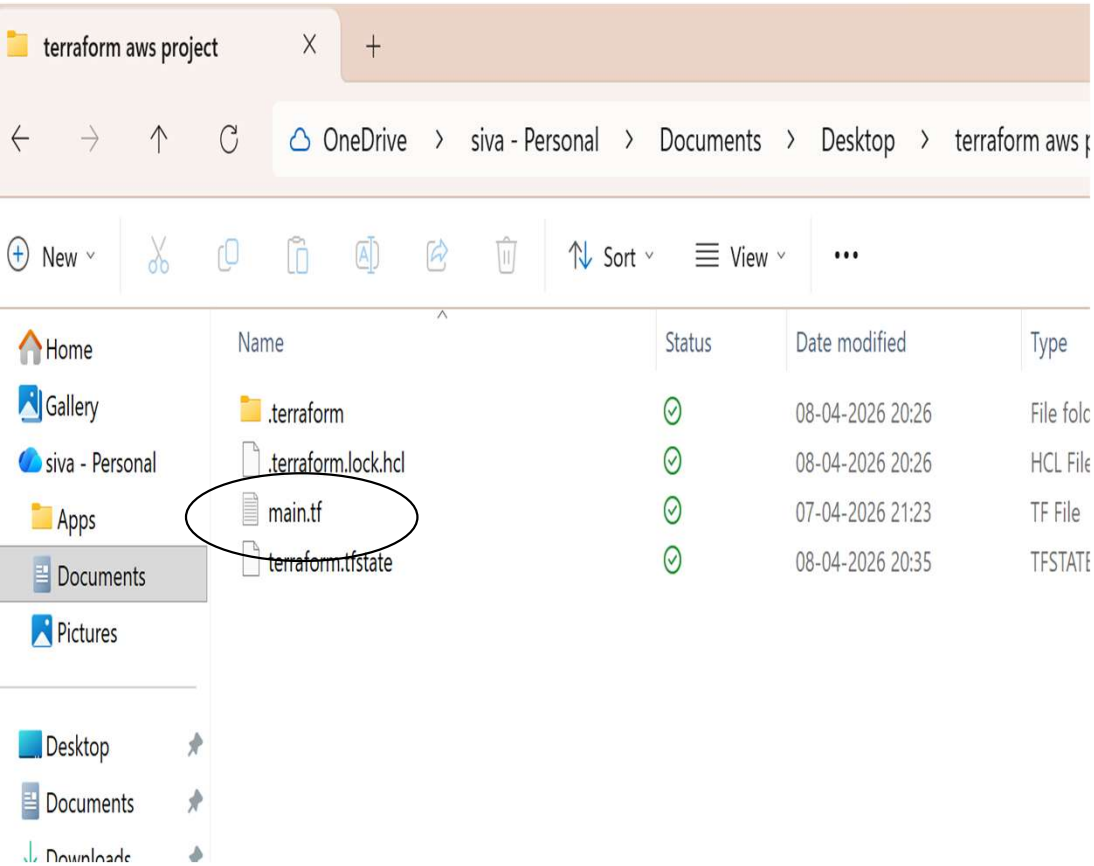
Terraform infrastructure Automation Project

Technologies : Terraform, AWS,GitHub

Main.tf

The main.tf file contains the core Terraform configuration that defines and provisions AWS infrastructure resources. It automates the creation of a complete cloud environment including networking and compute services.



In this project, the main.tf file is used to:

1.Create a custom Virtual Private Cloud (VPC)

2.Configure a public subnet for hosting resources

3.Attach an Internet Gateway to enable internet access

4.Set up route tables and associate them with the subnet

5.Define security groups to allow SSH (port 22) and HTTP (port 80) traffic

6.Dynamically fetch the latest Amazon Linux AMI

7.Launch an EC2 instance within the configured subnet

8.Assign a public IP address to the EC2 instance

9.Deploy a web server (Nginx) to host a simple website

10.This file demonstrates Infrastructure as Code (IaC) by automating the provisioning and configuration of cloud resources using Terrafo

To enable Terraform to interact with AWS services, the AWS Command Line Interface (CLI) was configured using secure access credentials.

Installed the AWS CLI on the local system
Generated Access Key ID and Secret Access Key from IAM user in AWS Management Console
Configured AWS CLI using the aws configure command
Set the default region and output format
Command used:
aws configure
Credentials provided:
AWS Access Key ID
AWS Secret Access Key
Default region (e.g., us-east-1)
Output format (json)

```
C:\Users\sivaa\OneDrive\Documents\Desktop\terraform aws project>aws configure
AWS Access Key ID [****************Y4UK]: AKIAV2FX6WNNKMQ2HUVU
AWS Secret Access Key [****************SVI/]: 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
Default region name [us-east-1]: us-east-1
Default output format [json]: json
```

Terraform Initialization

The terraform init command is used to initialize the working directory that contains Terraform configuration files. It prepares the environment before deploying infrastructure.

```
C:\Users\sivaa\OneDrive\Documents\Desktop\terraform aws project>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.39.0...
- Installed hashicorp/aws v6.39.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

C:\Users\sivaa\OneDrive\Documents\Desktop\terraform aws project>terraform validate
Success! The configuration is valid.
```

```
t>terraform plan

C:\Users\sivaa\OneDrive\Documents\Desktop\terraform aws project>terraform plan
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-0e10497160c48e829]

Terraform used the selected providers to generate the following execution plan. Resc
  + create

Terraform will perform the following actions:

  # aws_instance.web_server will be created
  + resource "aws_instance" "web_server" {
      + ami                                  = "ami-0e10497160c48e829"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = true
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
    }

Plan: 8 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ec2_public_ip = (known after apply)
```

Terraform Plan

The terraform plan command is used to preview the changes that Terraform will make before applying them. It helps in understanding what resources will be created, modified, or deleted.

This step performs the following actions:
Analyzes the current configuration files (.tf)
Compares them with the existing infrastructure (if any)
Generates an execution plan showing:
Resources to be created
Resources to be updated
Resources to be destroyed
Helps identify potential errors before deployment
This ensures safe and controlled infrastructure provisioning

```
>terraform apply
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.main_vpc: Creating...
aws_vpc.main_vpc: Creation complete after 5s [id=vpc-09d1f102779930a4c]
aws_internet_gateway.igw: Creating...
aws_route_table.public_rt: Creating...
aws_subnet.public_subnet: Creating...
aws_security_group.ec2_sg: Creating...
aws_internet_gateway.igw: Creation complete after 2s [id=igw-0f41084902c098fb7]
aws_route_table.public_rt: Creation complete after 2s [id=rtb-08bc0a8058c2c4bce]
aws_route.internet_access: Creating...
aws_route.internet_access: Creation complete after 2s [id=r-rtb-08bc0a8058c2c4bce1080289494]
aws_security_group.ec2_sg: Creation complete after 5s [id=sg-0831af699a5572a93]
aws_subnet.public_subnet: Still creating... [00m10s elapsed]
aws_subnet.public_subnet: Creation complete after 13s [id=subnet-051d39edb284d46e7]
aws_route_table_association.public_assoc: Creating...
aws_instance.web_server: Creating...
aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-0429adfc4b33
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Still creating... [00m20s elapsed]
aws_instance.web_server: Still creating... [00m30s elapsed]
aws_instance.web_server: Creation complete after 36s [id=i-0c63465d307e16ef0]

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:
```

Terraform Apply

The terraform apply command is used to execute the changes defined in the Terraform configuration files and provision the infrastructure on AWS.

During execution:
Terraform displays the execution plan (similar to terraform plan)
Prompts for confirmation before proceeding
After approval (yes), it:
Creates new resources (VPC, Subnet, EC2, etc.)
Configures networking components
Attaches security groups
Deploys the infrastructure automatically